Machine Images in Cloud Computing

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Explain what a machine image is and why it is essential for repeatable, automated cloud infrastructure.
- Describe the components of an Amazon Machine Image (AMI) and the differences between instance store-backed and EBS-backed AMIs.
- Walk through the full AMI lifecycle: create, register, copy, share, deprecate, disable, and deregister.
- Compare AWS AMIs, GCP custom images, and GCP machine images and identify when to use each.
- Articulate the concept of a golden image and explain why it matters for security, compliance, and operational consistency.
- Use HashiCorp Packer to automate the creation of machine images as part of a CI/CD pipeline.
- Apply best practices for machine image management including versioning, tagging, security hardening, and lifecycle policies.

Why Machine Images Matter

The Problem: Manual Server Configuration

- In traditional operations, servers are provisioned and then configured by hand — installing packages, applying patches, editing configuration files
- This manual process is slow, error-prone, and produces inconsistencies between environments (dev, staging, production)
- When auto-scaling adds a new instance, that instance needs to be fully configured before it can serve traffic — manual steps introduce unacceptable delays
- The "snowflake server" problem: each server becomes unique, making it difficult to reproduce or debug issues

The Solution: Pre-Built Machine Images

- A machine image captures a complete, configured system — OS, installed software, configuration, and data — as a reusable template
- Launching an instance from a machine image produces an identical copy of that configured system every time
- This enables immutable infrastructure: instead of modifying running servers, you build a new image and replace the old instances
- Machine images are the bridge between Infrastructure as Code (defining what to build) and running compute workloads (the actual VMs)

Immutable Infrastructure vs. Mutable Infrastructure

- **Mutable infrastructure:** Launch a base OS, then use configuration management tools (Ansible, Chef, Puppet) to install and configure software on the running instance. Servers change over time.
- **Immutable infrastructure:** Build a complete machine image with all software pre-installed and configured. Launch instances from the image. Never modify running instances — if a change is needed, build a new image and replace instances.
- Most cloud-native teams use a hybrid approach: a custom machine image with the base OS and core dependencies baked in, plus lightweight configuration applied at boot via user data or a configuration management tool

Amazon Machine Images (AMIs)

What Is an AMI?

- An Amazon Machine Image (AMI) is a template that provides the information required to launch an EC2 instance
- You must specify a source AMI when you launch an instance
- You can launch multiple instances from a single AMI when you need multiple instances with the same configuration
- You can use different AMIs to launch instances when you need instances with different configurations

Components of an AMI

- **Root volume template:** The operating system, application server, and applications — essentially the boot disk contents
- **Launch permissions:** Control which AWS accounts can use the AMI to launch instances (private, shared with specific accounts, or public)
- **Block device mapping:** Specifies the EBS volumes and instance store volumes to attach to the instance when launched

AMI Sources

- **AWS-provided AMIs:** Official images maintained by Amazon (Amazon Linux 2023, Ubuntu, Windows Server, etc.) — regularly updated with security patches
- **AWS Marketplace AMIs:** Third-party images that may include commercial software with licensing fees
- **Community AMIs:** Shared publicly by other AWS users — use with caution since quality and security are not guaranteed
- **Custom AMIs:** Images you create yourself from configured EC2 instances — the focus of this lecture

HVM vs. PV AMIs

- **HVM (Hardware Virtual Machine):** Presented with a fully virtualized set of hardware and boot by executing the master boot record. Can take advantage of hardware extensions for enhanced networking and GPU processing. All current-generation instance types require HVM AMIs.
- **PV (Paravirtual):** Guest OS runs on hardware that does not provide full virtualization but uses special APIs (hypercalls). Legacy approach — not recommended for new workloads.
- AWS recommends HVM AMIs for all new instances

Root Device Volumes: Instance Store vs. EBS

Instance Store-Backed AMIs:

- The root volume is stored in instance store (ephemeral storage on the physical host)
- When launched, the image is copied from S3 to the root volume
- Data persists only while the instance is running — all data is lost if the instance is terminated or fails
- Instance store-backed instances do not support the Stop action — they can only be running or terminated
- AMI stored in Amazon S3; boot time is slower because the entire image must be copied from S3

EBS-Backed AMIs:

- The root volume is an Amazon EBS volume created from an EBS snapshot
- EBS volumes persist independently from the life of the instance
- An EBS-backed instance can be stopped and later restarted without losing data on the root volume
- AMI creation is simpler — the CreateImage API creates the snapshot and registers the AMI
- Boot time is faster because only the parts needed to boot must be retrieved from the snapshot
- The standard for virtually all modern EC2 instances

The AMI Lifecycle

Create

- Create an AMI from a running or stopped EC2 instance (recommended: stop the instance first for a consistent snapshot)
- The instance customizations — installed software, configuration changes, application code — are saved in the new AMI
- For EBS-backed instances, AWS creates a snapshot of each attached EBS volume

Register

- After creation, the AMI is registered and receives a unique AMI ID (e.g., ami-0abcdef1234567890)
- The AMI is now available to launch instances

Copy

- AMIs are regional resources — an AMI in us-east-1 cannot be used to launch instances in eu-west-1
- Use the CopyImage API to copy an AMI to another region for multi-region deployments
- Copying an AMI does not copy launch permissions, tags, or S3 bucket permissions — these must be set separately on the new AMI
- Copying an unencrypted AMI produces an unencrypted copy; you can encrypt the copy by specifying a KMS key during the copy operation

Share

- By default, an AMI is private to the owning account
- You can share an AMI with specific AWS accounts by modifying launch permissions
- You can make an AMI public so any AWS account can use it
- AMIs with encrypted volumes cannot be made public; they can only be shared with specific accounts
- Sharing does not transfer ownership — the original account remains the owner

Deprecate

- Mark an AMI as superseded or out of date by setting a deprecation date
- Deprecated AMIs are hidden from AMI listings but can still be used if the AMI ID is known
- Users and services (e.g., launch templates, auto-scaling groups) can continue to launch instances from deprecated AMIs

Disable

- Temporarily prevent an AMI from being used to launch new instances
- If re-enabled, the AMI can be used again
- Does not affect instances already launched from the AMI

Deregister (Delete)

- Permanently deregister an AMI so it can no longer be used
- Deregistering does not delete the underlying EBS snapshots — these must be deleted separately to avoid storage charges
- Does not affect running instances previously launched from the AMI

Google Cloud Image Resources

GCE Image Types

Google Cloud has three distinct image-related resources. Understanding when to use each is important.

1. **Public OS Images:**
   - Provided and maintained by Google, open-source communities, and third-party vendors
   - All Google Cloud projects have access to public images by default
   - Include standard OSes: Debian, Ubuntu, CentOS, RHEL, SUSE, Windows Server, and more
   - Most public images are free; premium images (RHEL, SLES, Windows) add licensing charges
   - Grouped into image families (e.g., debian-12 in the debian-cloud project) — always points to the latest non-deprecated version
2. **Custom OS Images:**
   - Images you create and own, available only within your project (unless shared)
   - Can be created from a boot disk of an existing instance, from another image, from a snapshot, or imported from an on-premises virtual disk
   - Capture the contents of a single disk (typically the boot disk)
   - Support image families for versioning — the family always points to the latest, non-deprecated image in the family
   - Incur an image storage charge
3. **Machine Images (Compute Engine Resource):**
   - A more comprehensive resource that stores all configuration, metadata, permissions, and data from multiple disks of a VM instance
   - Unlike custom images (which capture a single disk), machine images capture the entire VM: instance properties, attached disks, network configuration, labels, tags, and metadata
   - Use incremental, differential snapshots for efficient storage
   - Guarantee crash-consistency across all attached disks at a given point in time
   - Stored in Cloud Storage (multi-regional or regional, configurable for compliance)

When to Use Each GCE Resource

Use Case	Custom Image	Machine Image	Snapshot	Instance Template
Create instances with pre-configured OS/software	Yes (single disk)	Yes (full VM)	No	Partial (references an image)
Backup and restore a complete VM	No	Yes	Partial (single disk)	No
Instance cloning	No	Yes	No	No
Multi-disk consistency	No	Yes	No	No
Image family versioning	Yes	No	No	No
Auto-scaling / MIG	Via instance template	Via instance template	No	Yes (references image)

GCE Image Families

- An image family groups related custom images together and always points to the latest non-deprecated version
- When you reference an image family (instead of a specific image name) in a launch command or template, GCE automatically uses the most recent image
- You can roll back by deprecating the latest image, which causes the family to point to the previous version
- Useful for automating deployments — your Terraform or instance templates reference the family, not a specific image version

GCE Image Deprecation States

- **Active:** Default state; image is available for use
- **Deprecated:** Image is available but marked as outdated; a replacement image can be specified. Using a deprecated image logs a warning.
- **Obsolete:** Image is not available for new instance creation but existing instances are unaffected
- **Deleted:** Image is permanently removed

Azure Managed Images and Image Gallery

Azure VM Images

- **Azure Marketplace Images:** Pre-built images from Microsoft and partners (similar to AWS Marketplace AMIs)
- **Managed Images:** Captured from a generalized or specialized VM (similar to AMIs)
- **Azure Compute Gallery (formerly Shared Image Gallery):** A service for managing, sharing, and distributing VM images across subscriptions, tenants, and regions. Supports versioning, replication, and RBAC-based access control.
-

Cross-Provider Comparison

Cross-Provider Comparison

Feature	AWS (AMI)	GCP (Custom Image / Machine Image)	Azure (Managed Image / Gallery)
Scope	Regional	Global (custom images) / Configurable (machine images)	Regional (replicated via Gallery)
Multi-disk capture	Yes (block device mapping)	Custom Image: No (single disk); Machine Image: Yes	Managed Image: Yes; Gallery: Yes
Versioning	Manual (naming conventions)	Image families (built-in)	Gallery: built-in versioning
Sharing across accounts	Launch permissions	IAM permissions across projects	Gallery: RBAC + cross-tenant sharing
Encryption	KMS (default or CMK)	Google-managed, Cloud KMS, or CSEK	Azure Disk Encryption, CMK
Incremental/differential storage	EBS snapshots (incremental)	Machine images (differential)	Gallery (incremental replication)

Golden Images and Custom Image Strategy

What Is a Golden Image?

- A golden image (also called a base image, master image, or hardened image) is a carefully crafted machine image that serves as the canonical starting point for all instances in an organization
- It includes a stable, patched OS, security hardening, monitoring agents, logging configuration, and organizational standards
- Application-specific images are then built on top of the golden image by adding application code and dependencies
-

Why Golden Images Matter

- **Security and compliance:** Every instance starts from a known, hardened baseline. Security patches are applied once to the golden image, not individually to every running instance. Compliance audits can verify that all instances derive from an approved image.
- **Consistency:** Eliminates configuration drift — every instance launched from the same image is identical. Reduces "works on my machine" problems between dev, staging, and production.
- **Speed:** Auto-scaling launches a fully configured instance in seconds (just boot from image) rather than minutes (boot base OS, then run setup scripts). This directly impacts application responsiveness during traffic spikes.
- **Reliability:** Fewer moving parts at launch time — no dependency on external package repositories, artifact servers, or configuration management servers being available during instance startup.

The Golden Image Pipeline

A golden image pipeline is an automated workflow that builds, tests, and distributes machine images. A typical pipeline:

- **Source:** Start with an AWS-provided AMI or GCP public image (e.g., Amazon Linux 2023, Ubuntu 24.04)
- **Build:** Use Packer (or EC2 Image Builder / GCP Cloud Build) to launch a temporary instance, run provisioning scripts (install packages, apply patches, harden the OS, install agents), and create a new image
- **Test:** Automatically launch an instance from the new image and run validation tests — does it boot? Are required services running? Do security benchmarks (CIS, STIG) pass?
- **Approve:** Gate the pipeline for manual or automated approval
- **Distribute:** Copy the image to all required regions (AWS) or share across projects (GCP)
- **Register:** Update launch templates, instance templates, or Terraform variables to reference the new image
- **Retire:** Deprecate and eventually deregister old images per your retention policy

Golden Image Layering Strategy

- **Layer 0 — Provider base image:** Amazon Linux 2023, Ubuntu 24.04, etc.
- **Layer 1 — Organizational base image (Golden Image):** OS hardening (CIS benchmarks), security agent (e.g., CloudWatch agent, Datadog), log forwarding, SSH configuration, time sync
- **Layer 2 — Role-specific image:** Application runtime (Java, Node.js, Python), web server (Nginx, Apache), database client libraries
- **Layer 3 — Application image:** Your application code, configuration files, and dependencies

Each layer inherits from the one below. When a security patch is needed at Layer 1, you rebuild Layers 1 through 3 and roll out new instances.

Building Machine Images with Packer

What Is Packer?

- HashiCorp Packer is an open-source tool for creating identical machine images for multiple platforms from a single source configuration
- Packer is not a configuration management tool — it uses provisioners (shell scripts, Ansible, Chef, etc.) to install and configure software on a temporary instance, then captures the result as an image
- Supports multiple builders: Amazon EBS, Google Cloud, Azure, VMware, Docker, and many more
- Uses HCL2 (HashiCorp Configuration Language) templates

Packer Architecture

- **Templates (HCL2):** Define the image build configuration in .pkr.hcl files
- **Builders:** Responsible for creating the temporary instance and producing the final image. The Amazon EBS builder (amazon-ebs) is the most common for AWS.
- **Provisioners:** Scripts or tools that run on the temporary instance to install and configure software. Common provisioners include shell (bash scripts), ansible, file (copy files to the instance), and powershell (Windows).
- **Post-processors:** Optional steps that run after the image is created, such as creating a manifest file or uploading metadata.

Packer Template Example (AWS)

```
packer {
  required_plugins {
    amazon = {
      source  = "github.com/hashicorp/amazon"
      version = "~> 1"
    }
  }
}

source "amazon-ebs" "webapp" {
  ami_name      = "csye6225-webapp-{{timestamp}}"
  instance_type = "t2.micro"
  region        = "us-east-1"
  source_ami_filter {
    filters = {
      name                = "al2023-ami-*-x86_64"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["amazon"]
  }
  ssh_username = "ec2-user"
}

build {
  sources = ["source.amazon-ebs.webapp"]

  provisioner "shell" {
    scripts = [
      "scripts/update-os.sh",
      "scripts/install-app.sh",
      "scripts/cleanup.sh"
    ]
  }
}
```

Packer Build Workflow

- Packer reads the template and launches a temporary EC2 instance from the specified source AMI
- Packer connects to the instance via SSH (Linux) or WinRM (Windows)
- Provisioners run in order — installing packages, copying files, configuring services
- Packer stops the instance and creates an AMI from it (snapshot + register)
- Packer terminates the temporary instance and any temporary resources
- The resulting AMI ID is output and can be used in Terraform, launch templates, or CI/CD pipelines

Packer in CI/CD

- Packer builds can be triggered by a CI/CD pipeline (GitHub Actions, Jenkins, GitLab CI) whenever application code or infrastructure configuration changes
- The pipeline runs **packer init** (download plugins) → **packer validate** (check syntax) → **packer buil**d (create image)
- The output AMI ID is stored as an artifact and fed into the next stage (e.g., Terraform deployment)
- This creates a fully automated path from code commit to running infrastructure

Machine Image Management Best Practices

Naming and Tagging

- Use consistent naming conventions that encode useful information: {org}-{role}-{os}-{version}-{date} (e.g., csye6225-webapp-al2023-v1.2.3-20260211)
- Tag images with metadata: environment, owner, application, build number, git commit hash
- Tags enable automation: lifecycle policies can target images by tag, and cost allocation reports can track image storage costs

Versioning and Rollback

- Maintain at least the last 2–3 versions of each image for rollback capability
- In GCP, use image families to automatically point to the latest version; deprecate old versions
- In AWS, update launch templates to reference the new AMI; keep old launch template versions for rollback
- Test new images before promoting them to production

Security Hardening

- Start from a minimal base image — install only what is needed
- Apply all OS security patches before creating the image
- Remove default credentials, unnecessary users, and unused services
- Disable root SSH login; configure key-based authentication only
- Install and configure security monitoring agents
- Run automated security benchmarks (CIS, STIG) as part of the image build pipeline

Lifecycle Management and Cost Control

- Images and their underlying snapshots incur storage costs — old, unused images accumulate charges
- Implement retention policies: automatically deprecate images older than N days, deregister images older than M days
- Use AWS Data Lifecycle Manager or EC2 Image Builder lifecycle policies to automate cleanup
- When deregistering an AMI, remember to also delete the associated EBS snapshots — deregistering the AMI alone does not free storage
- Audit image usage: identify AMIs that no longer have running instances and are candidates for cleanup

Summary

Key Takeaways

- Machine images are the foundation of repeatable, automated cloud infrastructure. They eliminate configuration drift and enable rapid, consistent instance launches.
- In AWS, the AMI is a regional resource containing a root volume template, launch permissions, and block device mappings. Understand the full lifecycle: create, copy, share, deprecate, disable, deregister.
- GCP offers three image resources (custom images, machine images, snapshots) with distinct use cases. Machine images are the most comprehensive, capturing the full VM state including multiple disks.
- Golden images establish a secure, compliant, and consistent baseline. A layered strategy (base → organizational → role → application) simplifies maintenance and patching.
- Packer automates image creation and integrates into CI/CD pipelines. This is a core tool in your cloud engineering workflow and will be used in course assignments.
- Machine image management is a real operational responsibility. Naming, tagging, versioning, security hardening, and lifecycle policies are not optional — they are essential for a production-grade cloud environment.

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS